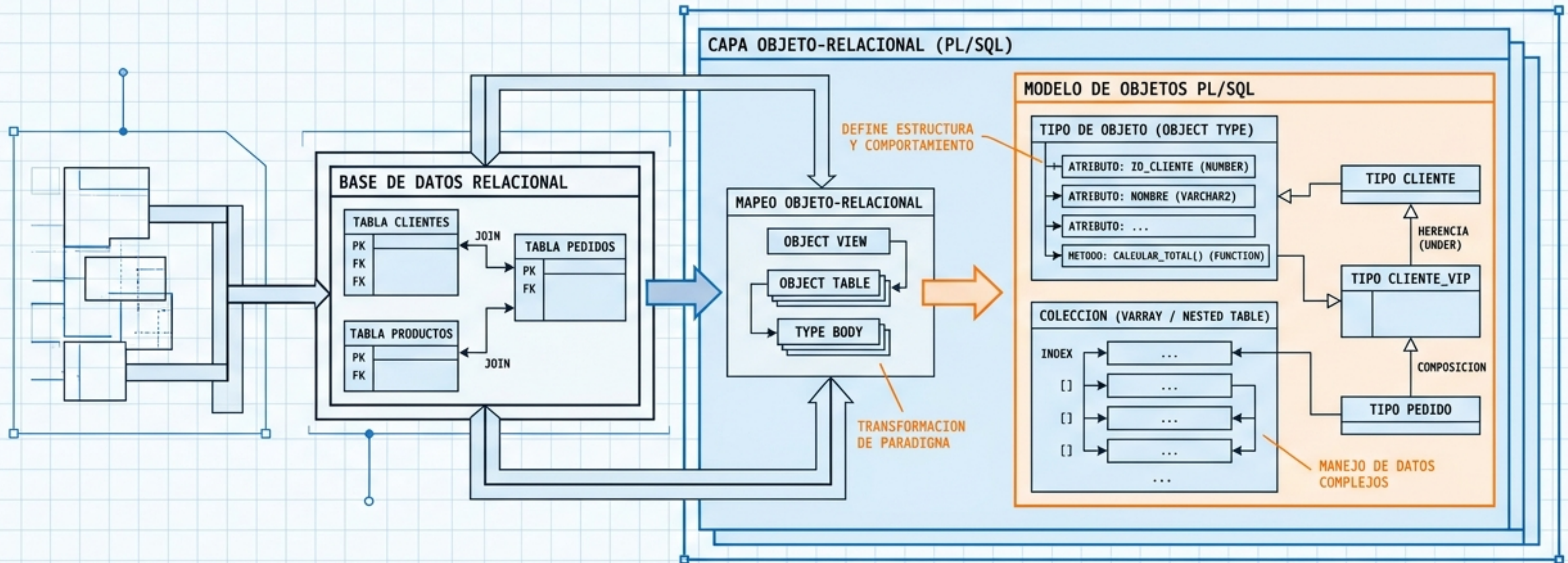


Arquitectura de Datos Objeto-Relacionales

// Del paradigma relacional al modelado de objetos en Oracle PL/SQL



[AZUL]: Estructura & Relación

[NARANJA]: Sintaxis Crítica & Callouts

[NEGRO]: Terminología General

[---]: Mapeo/Flujo

[<>]: Composición/Agregación

Documento de
Referencia Técnica

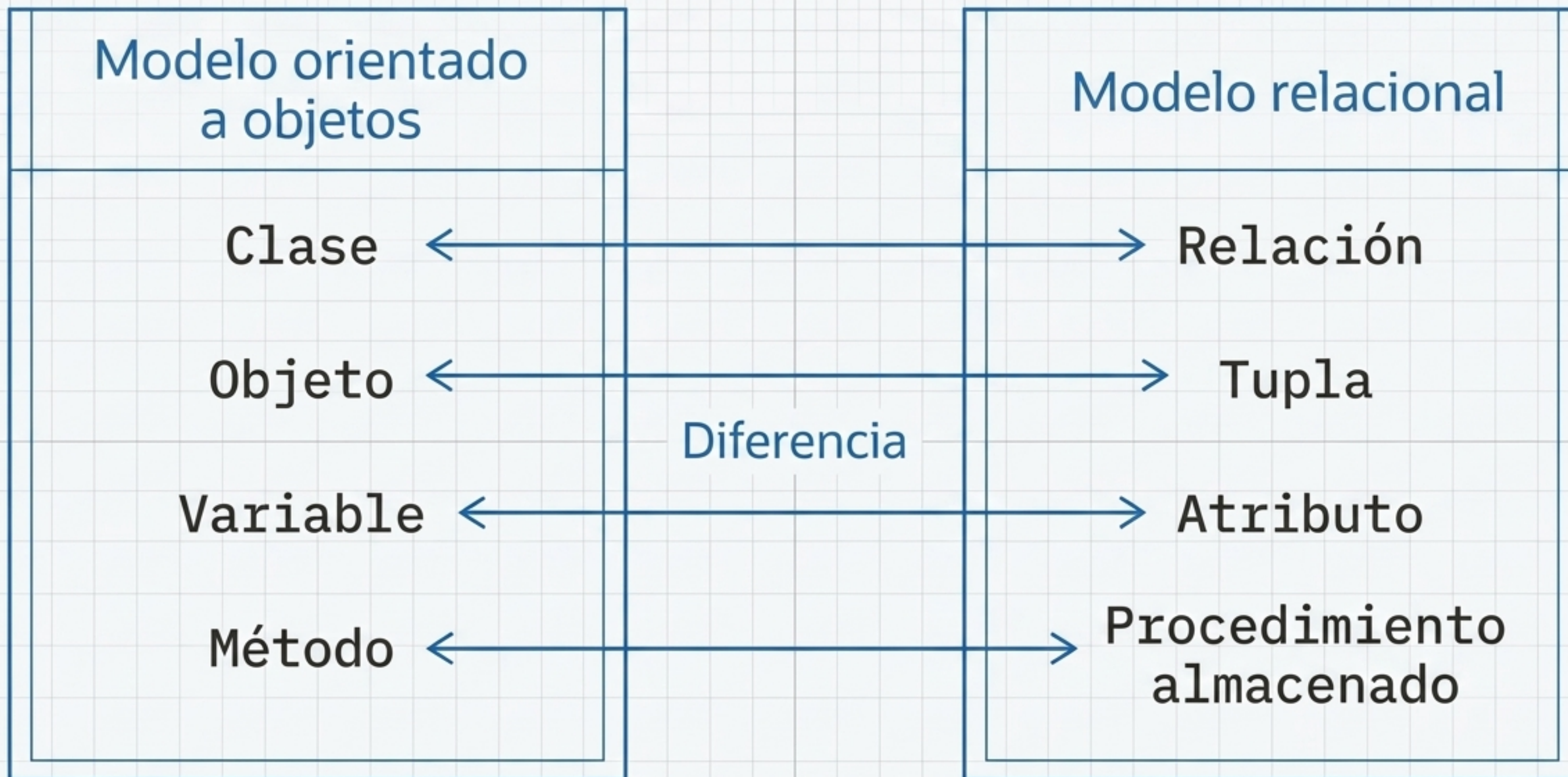
Base de Datos Relacional

- Tipos de datos simples y estructurados.
- Lenguajes de consulta y seguridad altamente robustos.
- Excelente para escenarios transaccionales tradicionales.

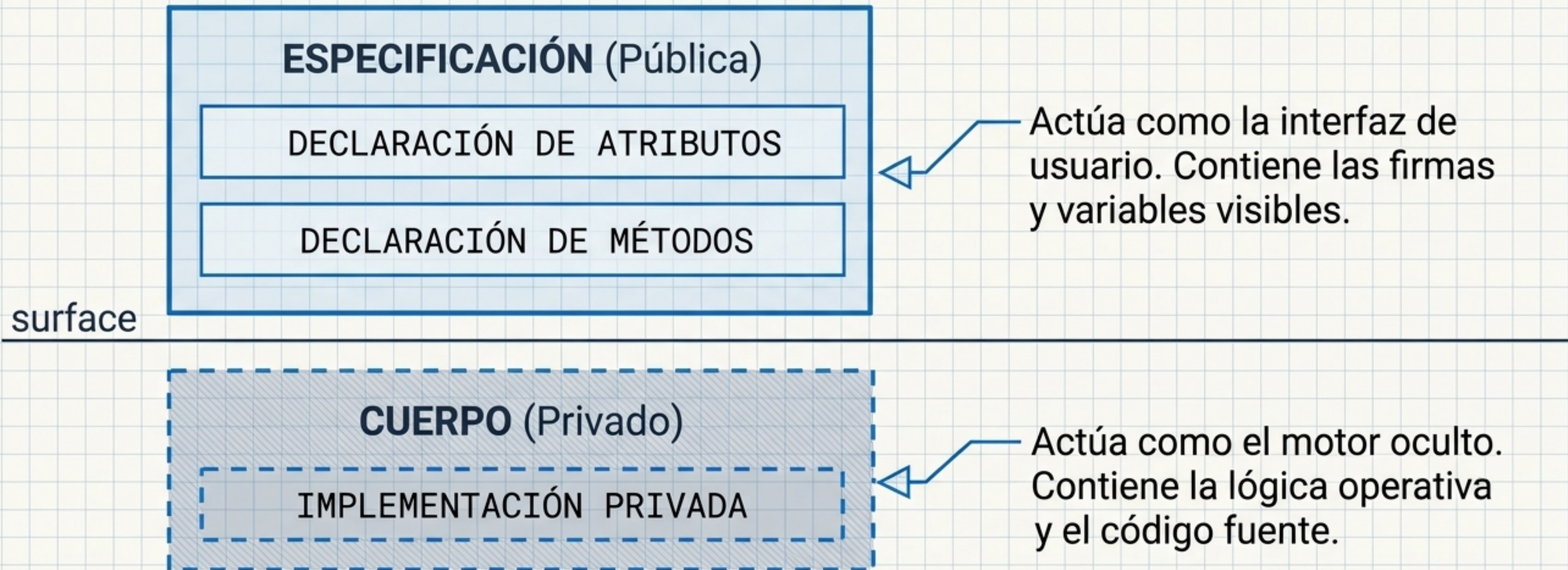
Base de Datos Objeto-Relacional (BDOR)

- Tipos de datos complejos y definidos por el usuario.
- Integración fluida con lenguajes de programación (**Java, PL/SQL**).
- Alto rendimiento y productividad para sistemas avanzados.
- Trade-off: Mayor coste y complejidad arquitectónica.

Síntesis: Las BDOR fusionan la solidez del modelo relacional con la flexibilidad estructural de la programación orientada a objetos.



Nota: En Oracle, esta correspondencia permite diseñar bases de datos híbridas donde la semántica del código refleja directamente el modelo de datos.



Regla de Encapsulación: La manipulación de datos se realiza exclusivamente a través de los métodos. Esto aísla el código interno de las aplicaciones cliente.

Ciclo de Vida

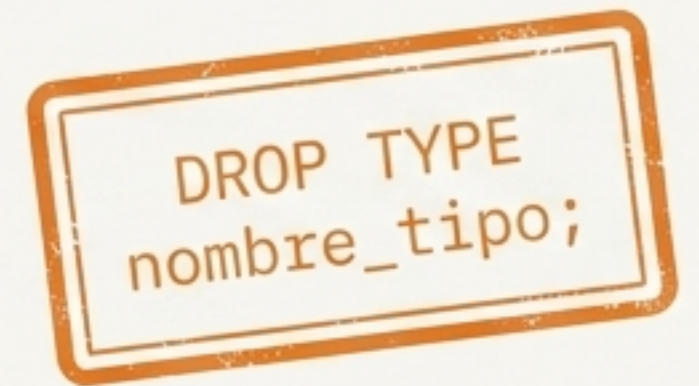
```
CREATE TYPE alumno AS OBJECT (  
    nombre VARCHAR2(50),  
    apellidos VARCHAR2(100),  
    dni VARCHAR2(9),  
    calificación NUMBER  
);
```

◇ **Orden Estricto:** Los atributos se declaran siempre antes que los métodos.

▷ **Prohibiciones:** No se pueden declarar constantes, excepciones, cursores o tipos dentro de la especificación.



(Action: Reemplazar existente)



(Action: Eliminar del sistema)

Declaración de Atributos: Restricciones de Sistema

Tipos Soportados

La mayoría de los tipos de datos estándar de Oracle (VARCHAR2, NUMBER, DATE, etc.).

LONG, LONG RAW

NCHAR, NCLOB, NVARCHAR2

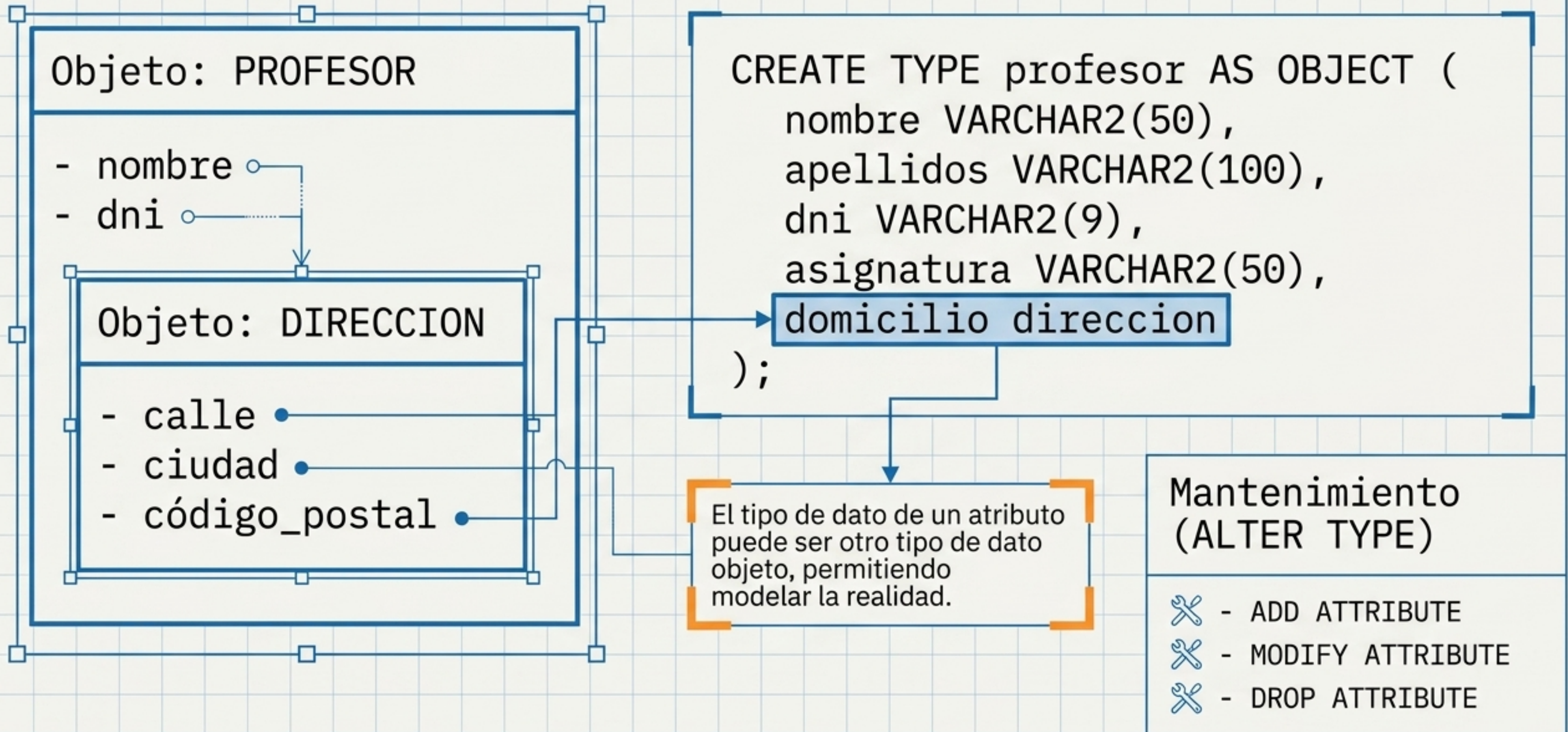
MLSLABEL, ROWID

Tipos PL/SQL:
BINARY_INTEGER, BOOLEAN,
%TYPE, %ROWTYPE



Alerta Crítica de Modelado: Está estrictamente prohibida la asignación de la restricción **NOT NULL** en los atributos de un tipo objeto.

Jerarquías Complejas: Objetos Anidados



Implementación de Métodos: Especificación e Interfaz vs. Cuerpo

Especificación / Interfaz

```
CREATE TYPE profesor AS OBJECT (  
  ...  
  MEMBER FUNCTION edad RETURN NUMBER,  
  MEMBER PROCEDURE aumentoSalario(a NUMBER)  
);
```

Enlace de Implementación

Cuerpo / Implementación

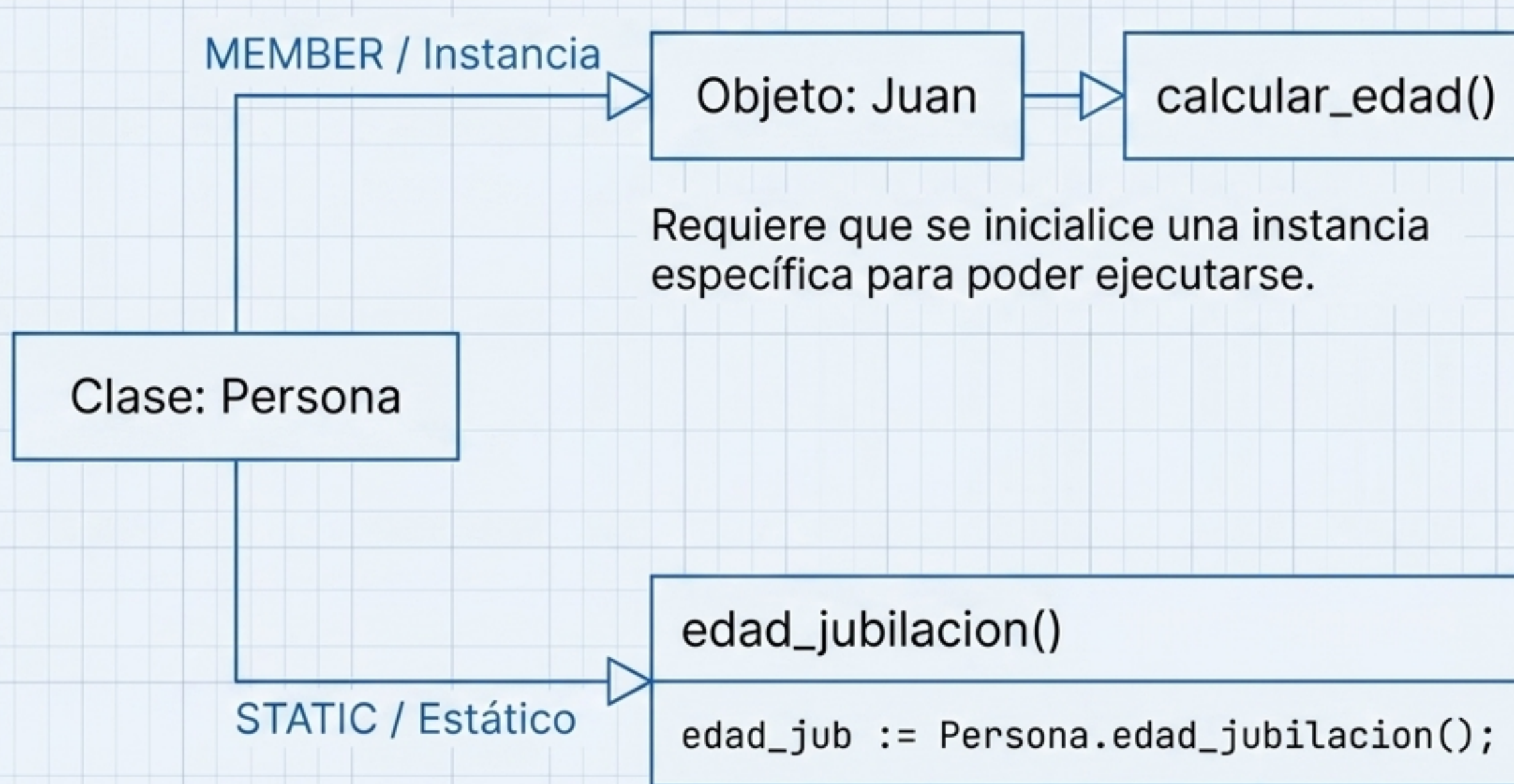
```
CREATE TYPE BODY profesor AS  
  MEMBER PROCEDURE aumentoSalario(a NUMBER) IS BEGIN  
    salario := salario + a;  
  END aumentoSalario;  
  ...  
END;
```

Clave de Métodos

- **MEMBER FUNCTION:**
Ejecuta lógica y obligatoriamente devuelve un valor (RETURN).
- **MEMBER PROCEDURE:**
Ejecuta una acción sobre el objeto, no devuelve valor.

⚠ **Regla:** El nombre del método NO puede ser igual al del objeto o sus atributos (excepto en los constructores).

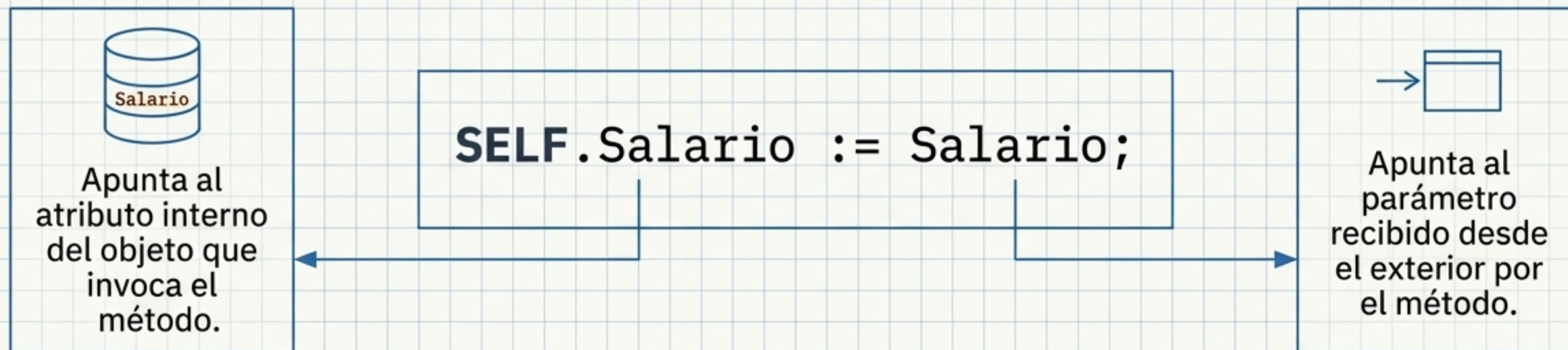
Invocación de Métodos: Instancia vs. Clase



Limitación Arquitectónica

Los métodos STATIC son operaciones generales. Al no depender de un objeto activo, **NO** pueden acceder ni modificar los atributos internos de ninguna instancia.

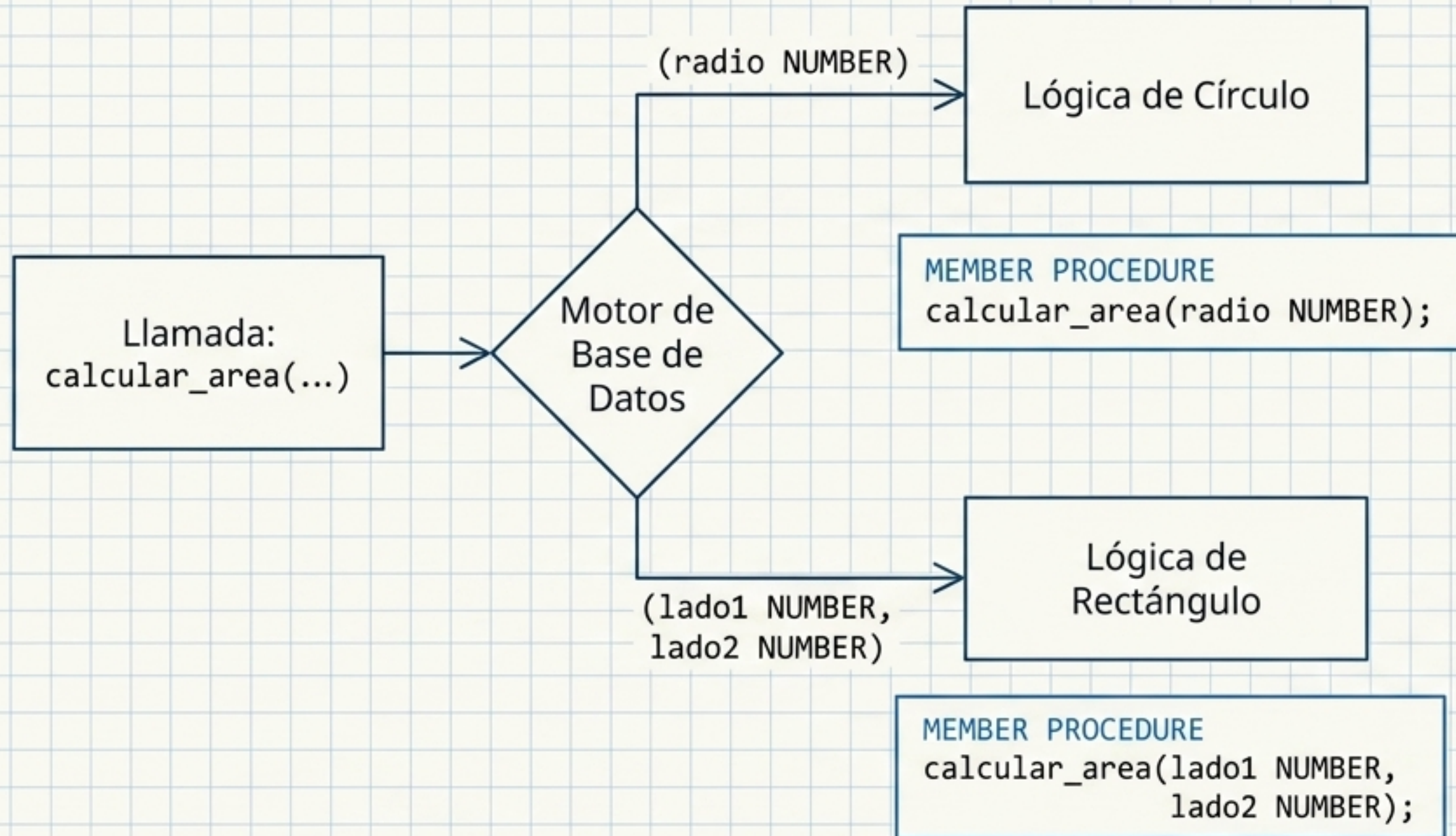
El Parámetro Implícito: SELF



Comportamiento por Defecto en Oracle	
En FUNCIONES	En PROCEDIMIENTOS
Modo IN. Solo lectura, no permite modificación del objeto.	Modo IN OUT . Permite tanto la lectura como la modificación de los atributos.

* Es el primer parámetro de todo método. El motor lo inyecta automáticamente aunque no se declare.

Mecánica de Enrutamiento: Sobrecarga (Overloading)



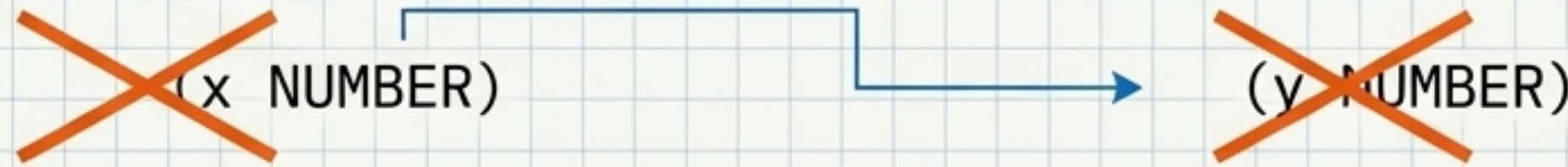
Condiciones Válidas para la Sobrecarga:

1. Diferente número de **parámetros**.
 - ...
2. Diferente **tipo de datos** de los parámetros.
3. Diferente **orden** en la firma.
 - ...

Antipatrones de Sobrecarga

Fallo 1: Diferencia solo en nombres

No se permite si solo cambian los nombres de las variables pero la firma tipada es idéntica. El motor no puede distinguirlos.



Fallo 2: Diferencia solo en tipo de retorno

Funciones con parámetros idénticos pero distinto tipo de RETURN generan un error de validación.

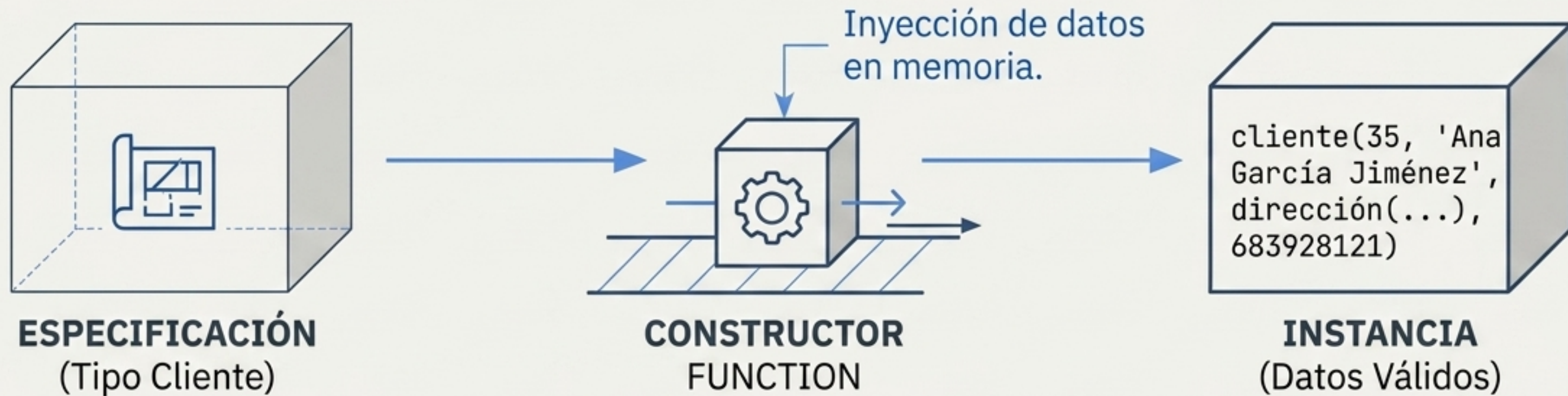
~~MEMBER FUNCTION obtener_datos(id NUMBER) RETURN VARCHAR2;~~
~~MEMBER FUNCTION obtener_datos(id NUMBER) RETURN DATE;~~

→ **VALIDATION ERROR**

Buenas Prácticas (Diseño Orientado al Mantenimiento):

Evitar el exceso de sobrecargas complejas; impactan negativamente en el rendimiento del sistema y dificultan la expansión a largo plazo.

Inicialización de Datos: Constructores



Comportamiento Nativo:

Todo objeto tiene un constructor por defecto con el mismo nombre que el tipo. Sus parámetros corresponden exactamente a los atributos.



Implementación Personalizada:

Es posible redefinir la inicialización usando la instrucción **CONSTRUCTOR FUNCTION** y finalizando con **RETURN SELF AS RESULT**.



Flexibilidad:

Los constructores soportan sobrecarga, permitiendo múltiples formas de inicializar el mismo objeto (ej. validaciones de saldo inicial).

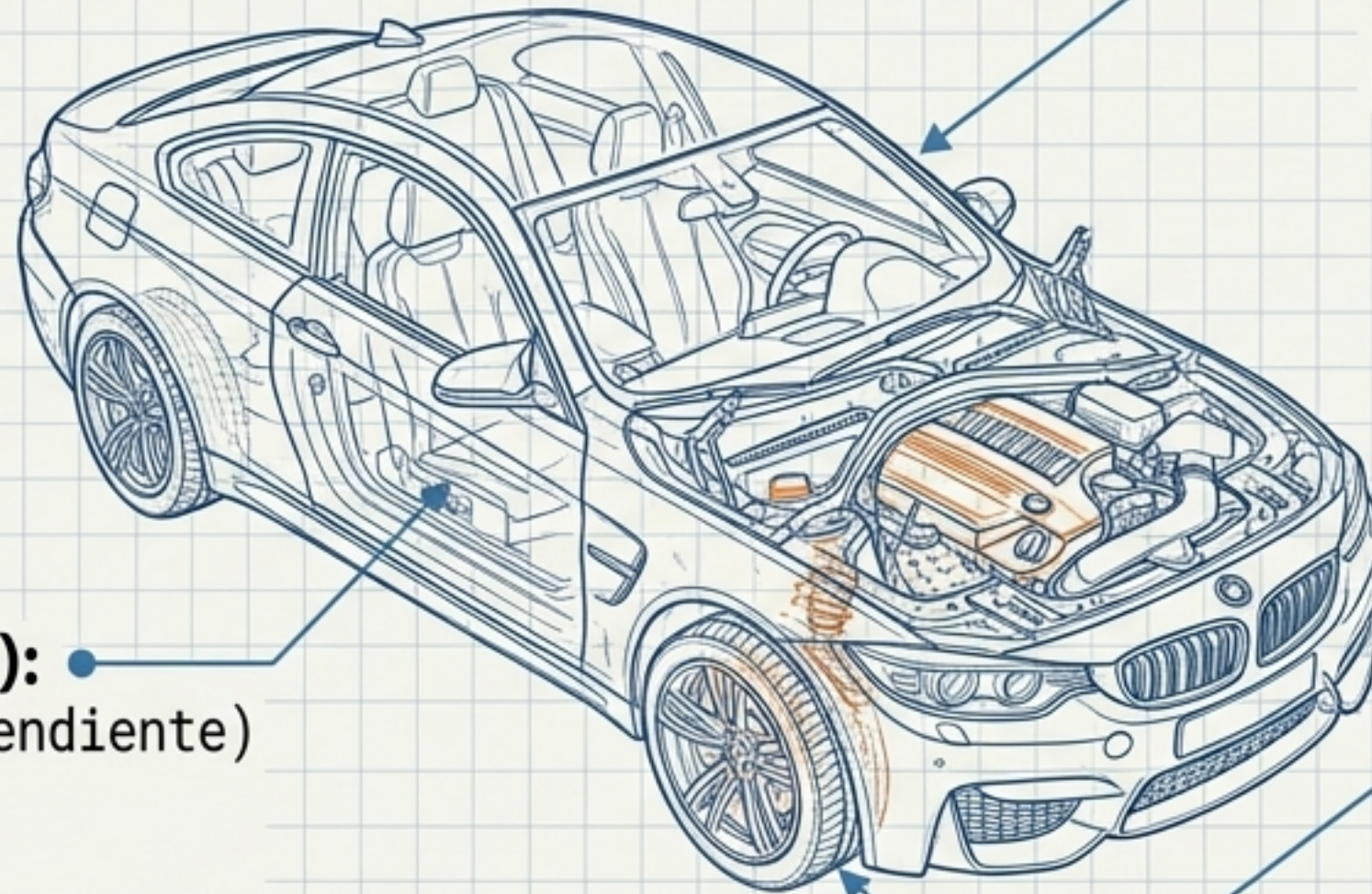
Síntesis del Modelado: El Objeto COCHE

Atributos Simples:

Color (VARCHAR2),
Fabricante (VARCHAR2)

Métodos Operativos:

calcular_consumo_medio(),
detectar_averias()



Atributo Anidado (Objeto):

Motor (Tipo Objeto Independiente)
-> atributos: potencia,
capacidad

Atributo Anidado (Objeto):

Rueda -> atributos:
presión, diámetro

Conclusión: El enfoque orientado a objetos permite modelar sistemas físicos complejos agrupando sus datos descriptivos y su lógica operativa en una única estructura autocontenida y reutilizable.

Hoja de Referencia Estructural PL/SQL

Ciclo de Vida del Tipo

CREATE TYPE nombre **AS OBJECT**
CREATE OR REPLACE TYPE
DROP TYPE nombre;

Estructura Interna

Atributos (Declarados primero)
MEMBER FUNCTION (Devuelve valor)
MEMBER PROCEDURE (Ejecuta acción)

Modificadores y Parámetros

STATIC (Método a nivel de clase)
SELF (Apunta a la instancia actual)
CONSTRUCTOR (Inicializa datos)

Mantenimiento Estructural

ALTER TYPE nombre
ADD ATTRIBUTE
MODIFY / DROP ATTRIBUTE

Las Bases de Datos Objeto-Relacionales proporcionan el marco estructural necesario para la ingeniería de datos moderna, segura y escalable en entornos Oracle.